CCA 3: COMPUTER VISION ASSIGNMENT

Practical Implementation & Project Documentation

Student Information	Details
Student Name	Sanket Salunkhe
PRN Number	1272250582
Course	Master of Computer Applications (MCA)
Subject	Computer Vision
Primary Artifact	CCA_3_Computer_Vision_Assignment.ipynb
Execution Environment	Google Colab
Programming Language	Python
Submission Date	October 2026

Executive Project Overview

This report documents the practical implementation of four computer vision areas: (1) Image Processing and Segmentation, (2) Feature Extraction and Matching, (3) Object Detection, and (4) Video Motion Estimation. The work was carried out in a Google Colab notebook using Python and computer vision libraries.

The notebook uses fruit images, a face photograph and a traffic video as input media. The documented outputs include image-processing visualizations, feature-analysis results, face and vehicle detection, extracted video frames, motion masks and optical-flow tracking.

Module	Implemented Techniques	Observed Output / Evidence
Task 1: Image Processing & Segmentation	Noise handling, spatial filtering and image segmentation techniques.	Fruit-image processing outputs to be inserted from Colab.
Task 2: Feature Extraction & Matching	HOG feature extraction and SIFT keypoint detection/matching.	Feature visualizations and matching outputs to be inserted from Colab.
Task 3: Object Detection	Viola-Jones Haar Cascade and YOLOv8.	1 face detected; YOLOv8 detected 6 objects in the traffic frame.
Task 4: Motion Estimation	Video frame extraction, frame differencing, Lucas-Kanade optical flow and GMM/MOG2 background subtraction.	782 video frames reported; 780 frames extracted; 100 feature points tracked.

1. Task 1: Image Processing and Segmentation

Objective: To apply image-processing operations to the supplied fruit image, study the effect of noise and filtering, and understand how segmentation techniques separate meaningful regions from the background.

1.1 Input Image Inspection

The original fruit image was loaded in Google Colab using OpenCV. The image was used as the reference input for the subsequent processing operations. The notebook workspace contains the files fruits_original.jpeg and fruits_second.jpeg.



Figure 1.1: Original input fruit image (fruits_original.jpeg)

1.2 Noise Modelling

Noise modelling introduces artificial disturbances into an image so that restoration methods can be evaluated. Gaussian noise represents random intensity variation, while salt-and-pepper noise introduces isolated bright and dark pixels.

- Gaussian noise: random variations are added to pixel intensities.
- Salt-and-pepper noise: selected pixels are replaced by black or white values.

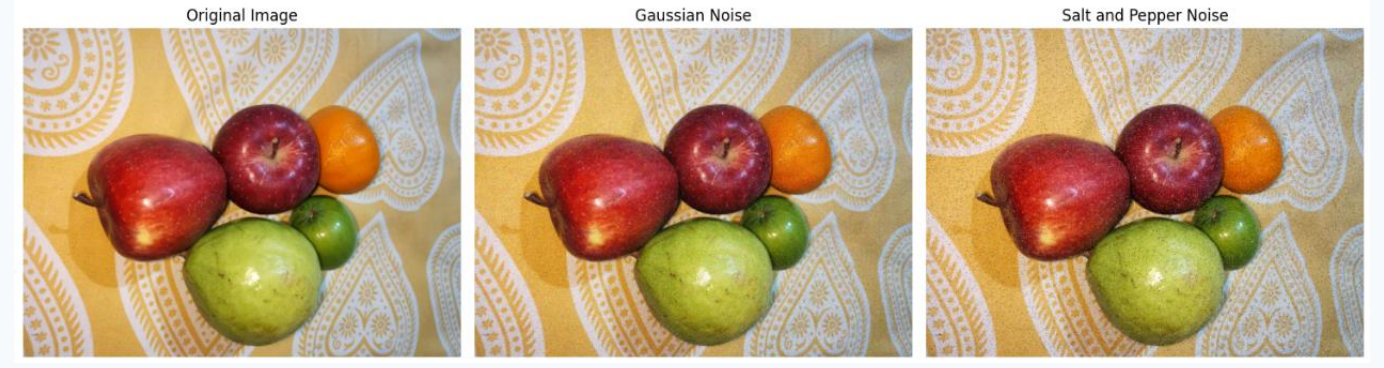


Figure 1.2: Original image and noisy image comparison

1.3 Spatial Filtering

Spatial filters operate on a pixel and its local neighbourhood. Gaussian filtering smooths an image using a weighted kernel. Median filtering replaces a pixel with the median of its neighbourhood and is commonly used to reduce impulse noise.

```
gaussian_result = cv2.GaussianBlur(image, (5, 5), 0)
median_result = cv2.medianBlur(noisy_image, 5)
```



Figure 1.3: Gaussian and median filtering output comparison

1.4 Image Quality Evaluation

Restoration quality can be evaluated by comparing a processed image with the original reference image. Peak Signal-to-Noise Ratio (PSNR) expresses reconstruction quality in decibels, while Structural Similarity Index (SSIM) compares structural information, contrast and luminance.

```
from skimage.metrics import peak_signal_noise_ratio, structural_similarity

psnr_value = peak_signal_noise_ratio(original, restored, data_range=255)
ssim_value = structural_similarity(
    original, restored, channel_axis=2, data_range=255
)
```

	Technique	PSNR (dB)	SSIM
0	Gaussian Noise	20.42	0.2528
1	Median on Gaussian	30.10	0.7083
2	Gaussian Filter on Gaussian	30.50	0.7241
3	Salt & Pepper Noise	18.96	0.3807
4	Median on Salt & Pepper	35.07	0.8880
5	Gaussian Filter on Salt & Pepper	29.05	0.6991

PSNR and SSIM calculations completed!

Figure 1.4: PSNR / SSIM values or comparison output from Colab

1.5 Image Segmentation

Image segmentation divides an image into meaningful regions. The Watershed method treats image intensity as a surface and uses markers to identify regions. It is useful when objects touch or overlap and need to be separated.

- Prepare a binary image and perform morphological operations where required.
- Identify foreground and background regions.
- Generate markers and apply the Watershed algorithm.
- Visualize the segmented regions and boundaries.

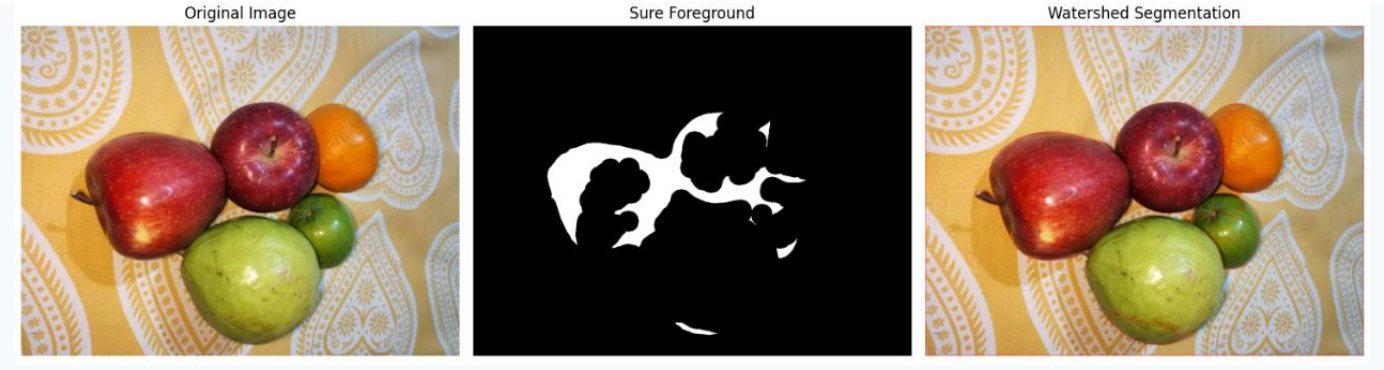


Figure 1.5: Watershed segmentation result

2. Task 2: Feature Extraction and Matching

Objective: To extract useful visual information from images using HOG and identify corresponding keypoints using SIFT. The experiment demonstrates how image features can be represented and matched.

2.1 Histogram of Oriented Gradients (HOG)

Histogram of Oriented Gradients is a feature descriptor that represents an image through the distribution of gradient directions. The image is divided into small cells, and gradient orientations are summarized into histograms. HOG is useful for describing object shape and appearance.

- Convert the input image into a suitable format.
- Calculate gradient magnitude and orientation.
- Divide the image into cells and build orientation histograms.
- Display the HOG visualization and extracted feature representation.



Figure 2.1: Input image and HOG feature visualization

2.2 SIFT Keypoint Detection

Scale-Invariant Feature Transform (SIFT) detects distinctive keypoints in an image and computes a descriptor for each keypoint. The method is designed to identify similar image regions even when scale or rotation changes.

- Detect keypoints in the input images.
- Compute descriptors for the detected keypoints.
- Display the keypoints overlaid on the image.



Figure 2.2: SIFT keypoints detected on the input image

2.3 Feature Matching

Feature matching compares descriptors from two images to identify corresponding visual features. A Brute-Force Matcher can compare descriptor vectors. Lowe's Ratio Test can be used to retain stronger matches and reject ambiguous pairs.

```
bf = cv2.BFMatcher()
matches = bf.knnMatch(des1, des2, k=2)

good_matches = []
for m, n in matches:
    if m.distance < 0.75 * n.distance:
        good_matches.append(m)
```



Figure 2.3: Feature matching between the two fruit images

2.4 Rotation and Scale Comparison

To examine geometric robustness, feature matching can be repeated after rotating or resizing an image. The resulting match visualizations show whether the same local image features can still be identified after transformation.

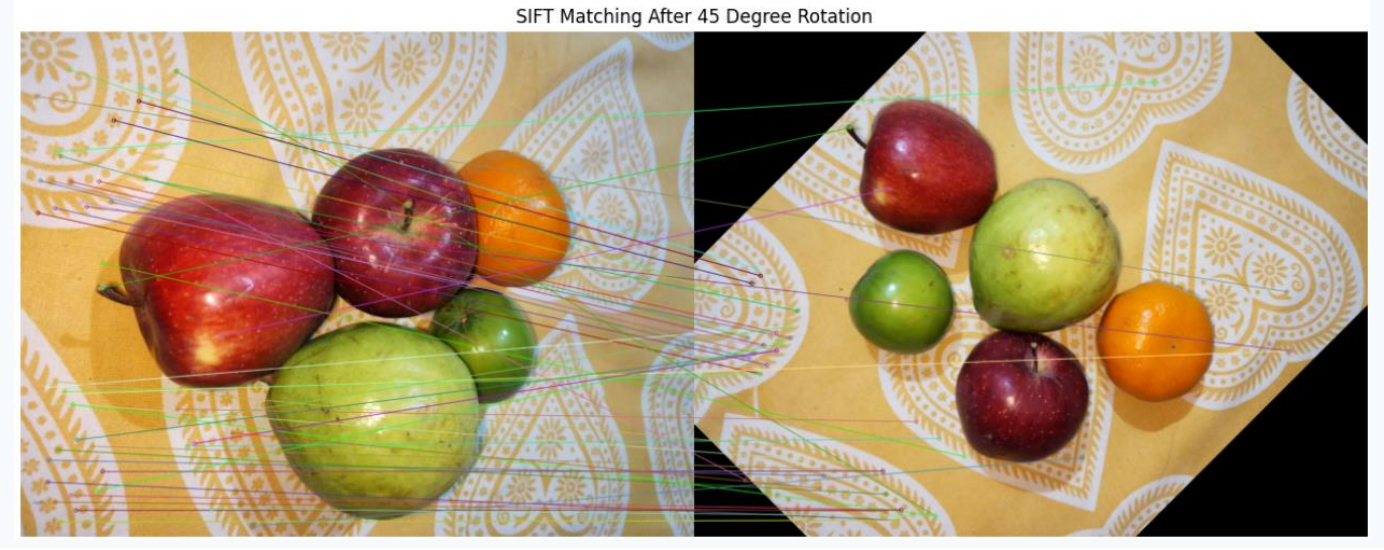


Figure 2.4: Feature matching after image rotation

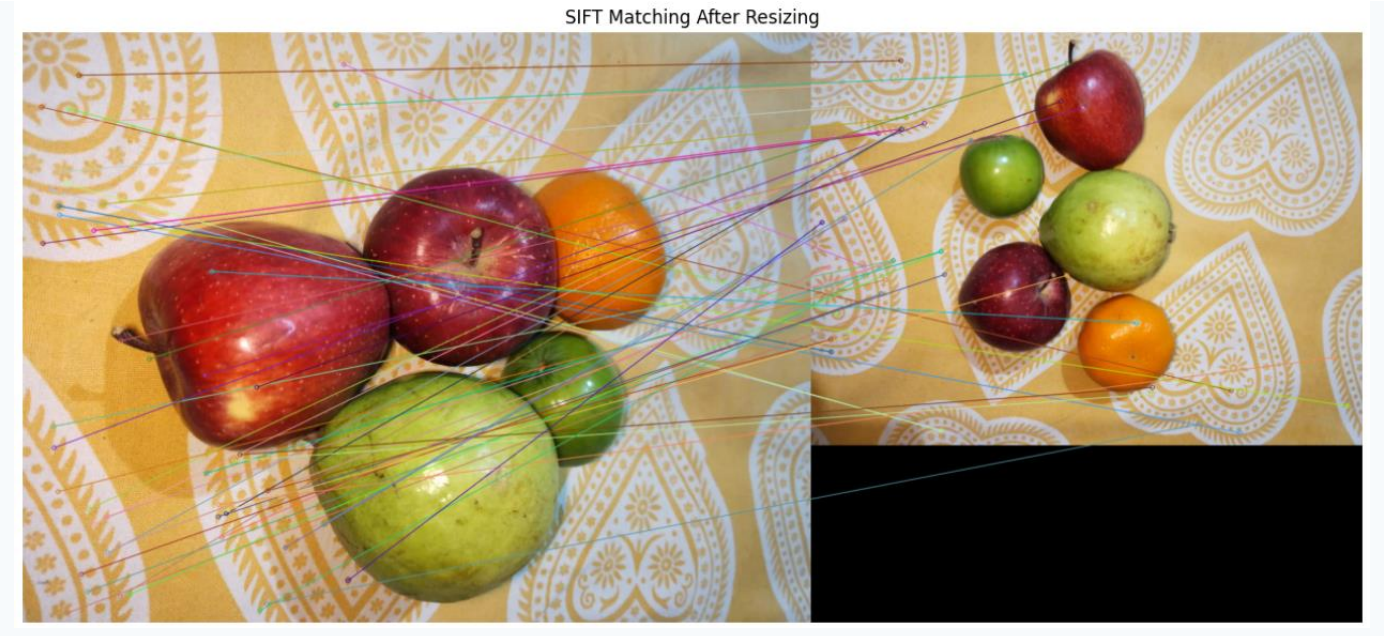


Figure 2.5: Feature matching after image resizing

3. Task 3: Object Detection

Objective: To implement and compare a classical face-detection approach (Viola-Jones Haar Cascade) with a deep-learning object-detection approach (YOLOv8).

3.1 Viola-Jones Face Detection

Viola-Jones is a classical object-detection method widely used for frontal face detection. The Haar Cascade classifier evaluates image regions using Haar-like features and a cascade of classifiers.

- Load the face photograph and convert it to grayscale.
- Load the pretrained frontal-face Haar Cascade classifier.
- Apply detectMultiScale() to locate face regions.
- Draw a rectangle around each detected face and display the result.

```
face_cascade = cv2.CascadeClassifier(
    cv2.data.haarcascades +
    "haarcascade_frontalface_default.xml"
)
faces = face_cascade.detectMultiScale(gray, 1.1, 5)
print("Faces detected:", len(faces))
```

Observed output: Faces detected = 1.



Figure 3.1: Viola-Jones face detection with bounding box

3.2 YOLOv8 Object Detection

YOLO (You Only Look Once) is a deep-learning object detector. It predicts object classes, bounding boxes and confidence scores. In this assignment, the pretrained YOLOv8n model was applied to a sample frame from the traffic video.

```
from ultralytics import YOLO

model = YOLO("yolov8n.pt")
results = model.predict(source=sample_frame, conf=0.25)
annotated_frame = results[0].plot()
```

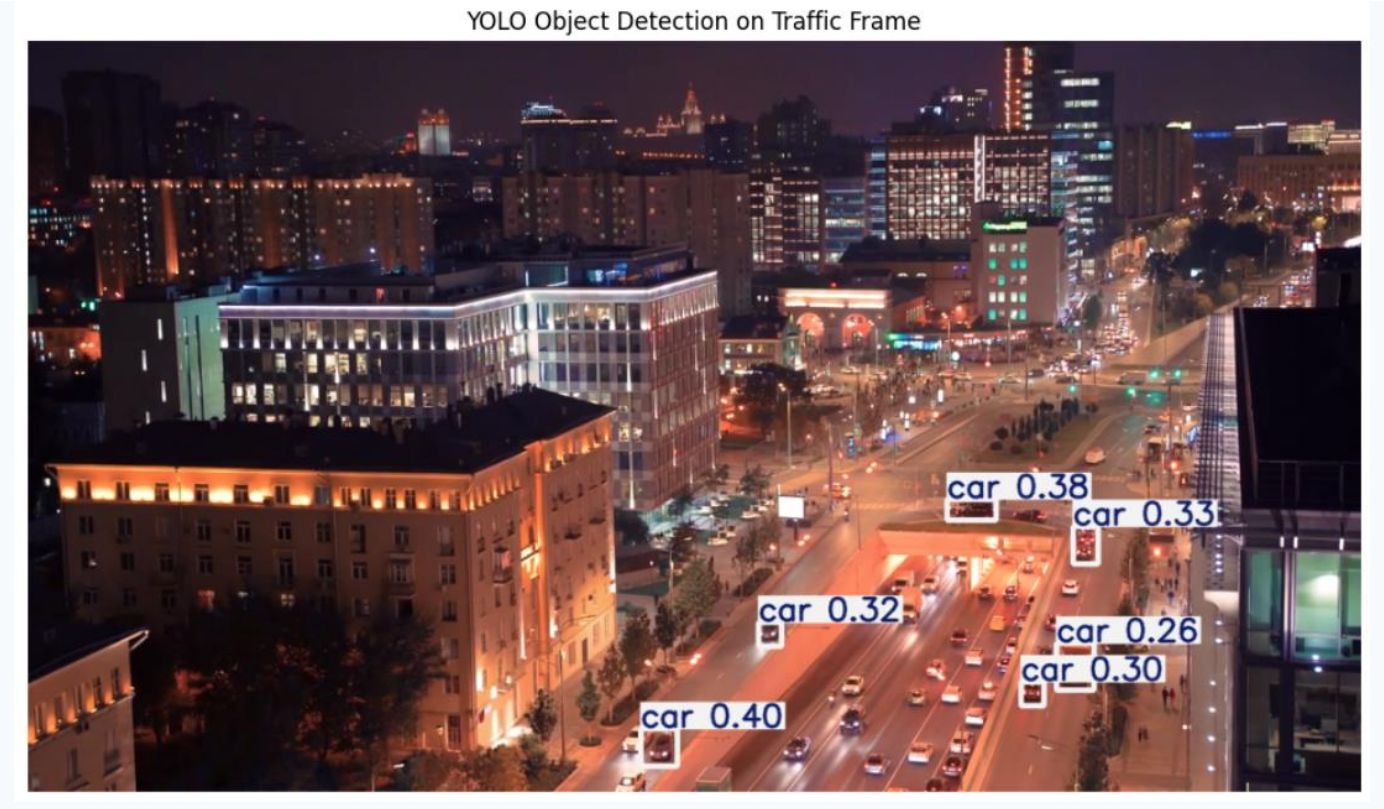


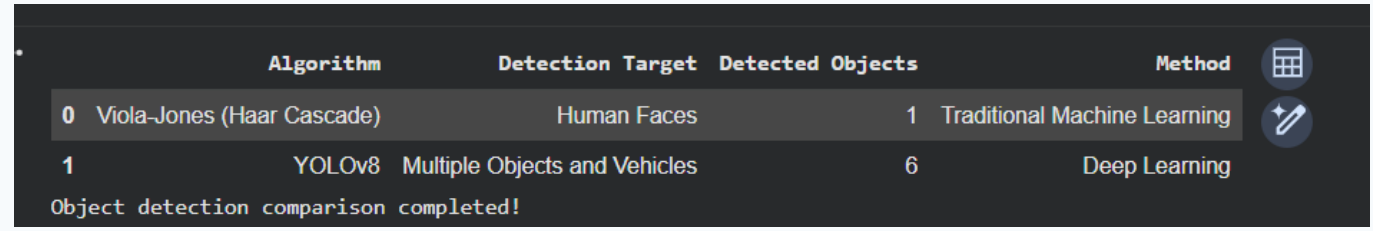
Figure 3.2: YOLOv8 object detection on the traffic frame

3.3 Object Detection Output Analysis

The displayed YOLOv8 output contains six detections, all labelled as cars. The confidence scores visible in the notebook output are listed below.

Detected Class	Confidence Score
Car	0.399
Car	0.383
Car	0.334
Car	0.317
Car	0.303
Car	0.257

These confidence scores represent the model's confidence for the corresponding detected bounding boxes. They are not overall model accuracy percentages.



	Algorithm	Detection Target	Detected Objects	Method
0	Viola-Jones (Haar Cascade)	Human Faces	1	Traditional Machine Learning
1	YOLOv8	Multiple Objects and Vehicles	6	Deep Learning

```
Object detection comparison completed!
```

Figure 3.3: YOLOv8 detection output and printed confidence scores

3.4 Comparison of Detection Methods

Parameter	Viola-Jones Haar Cascade	YOLOv8
Main purpose in this task	Frontal human face detection	General object detection
Detection result	1 face detected	6 car detections in the shown frame
Approach	Classical cascade classifier	Deep-learning object detector
Output	Face bounding box	Class labels, boxes and confidence scores

3.5 Result

Viola-Jones successfully detected one face in the uploaded photograph. YOLOv8 detected six cars in the selected traffic-video frame and displayed confidence scores for the detections.

4. Task 4: Video Motion Estimation

Objective: To analyze motion in a traffic video by extracting frames and applying frame differencing, Lucas-Kanade optical flow and Gaussian Mixture Model (GMM) background subtraction.

4.1 Video Stream Inspection

The traffic video was loaded using OpenCV VideoCapture. The notebook reported the following video properties:

Property	Observed Value
Frame rate	25.0 FPS
Total frames	782
Resolution	1280 × 720
Duration	31.28 seconds

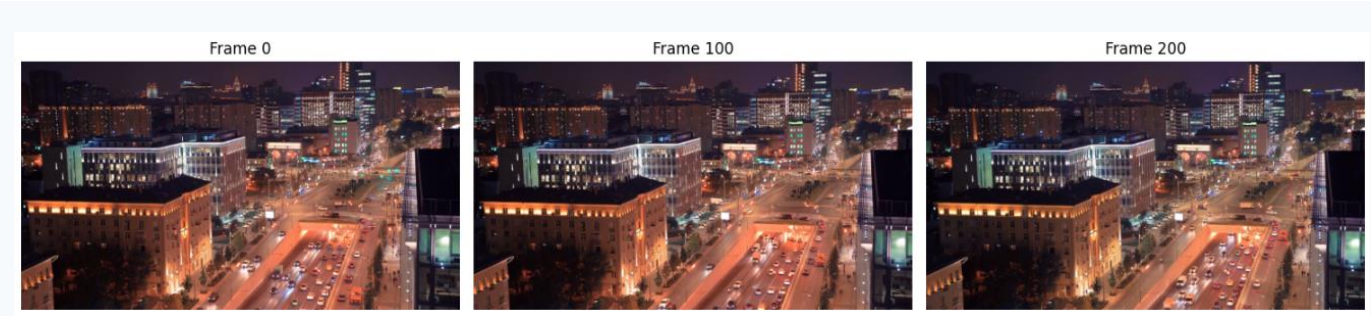


Figure 4.1: Sample traffic video frame

4.2 Video Frame Extraction

Video frame extraction reads a video sequentially and stores selected frames as images. The notebook output reported 780 extracted frames. Sample frames were displayed to verify the extraction process.

```
cap = cv2.VideoCapture(video_path)
frames = []

while True:
    ret, frame = cap.read()
    if not ret:
        break
    frames.append(frame)

cap.release()
print("Total extracted frames:", len(frames))
```

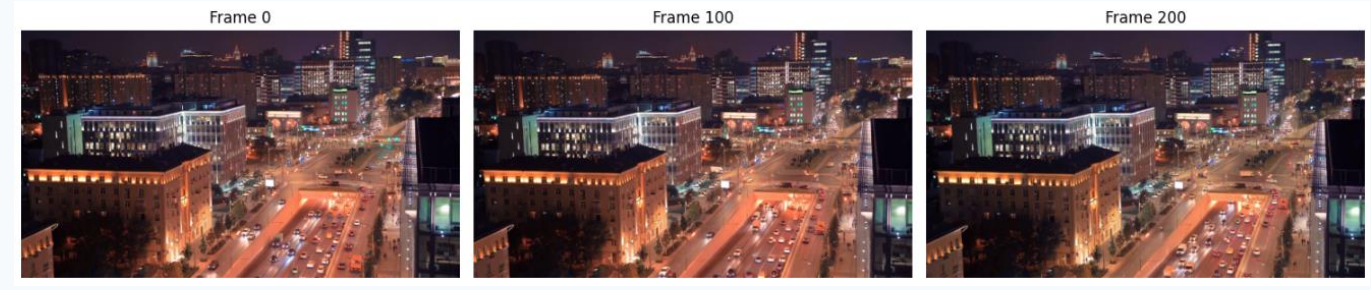


Figure 4.2: Extracted sample frames (Frame 0, Frame 100 and Frame 200)

4.3 Method 1: Frame Differencing

Frame differencing detects changes between consecutive frames. The frames are converted to grayscale and the absolute pixel-wise difference is calculated. Thresholding then creates a binary mask that highlights regions with sufficient change.

```
difference = cv2.absdiff(gray_frame_1, gray_frame_2)
_, foreground_mask = cv2.threshold(
    difference, 25, 255, cv2.THRESH_BINARY
)
```

The output screenshot shows the first frame, the second frame and the resulting frame-difference mask. The mask highlights changed regions between the two sampled frames.



Figure 4.3: Frame differencing mask generated from two traffic frames

4.4 Method 2: Lucas-Kanade Optical Flow

Lucas-Kanade optical flow estimates the movement of selected feature points between consecutive frames. The notebook detected 100 initial feature points and successfully tracked 100 points in the next frame.

- Detect trackable corners using a feature detector.
- Pass the previous and next grayscale frames to calcOpticalFlowPyrLK().
- Use the returned point locations and status values to identify successfully tracked points.
- Draw the tracked points or motion indicators on the frame.

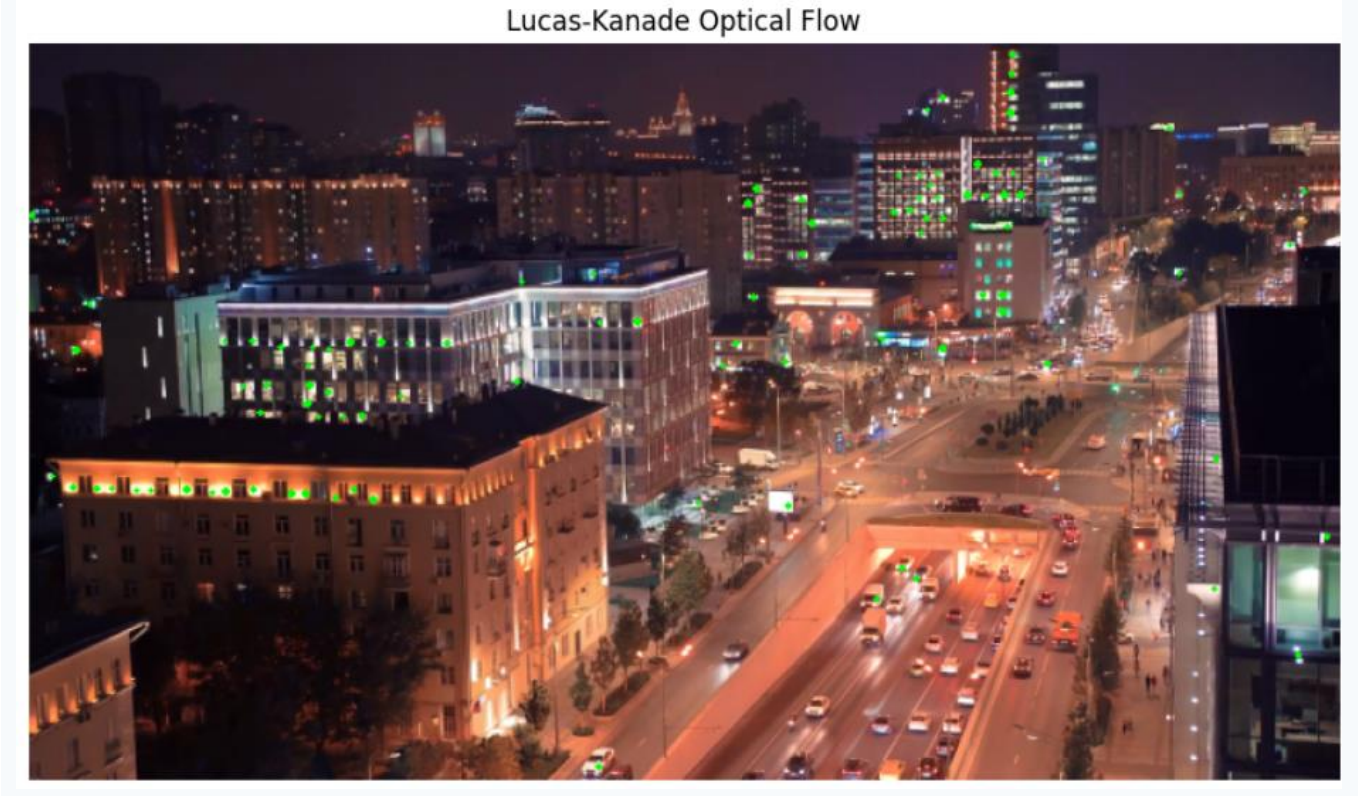


Figure 4.4: Lucas-Kanade optical flow with tracked feature points

4.5 Method 3: GMM Background Subtraction

Gaussian Mixture Model (GMM) background subtraction models the background at each pixel using a mixture of Gaussian distributions. OpenCV provides the MOG2 implementation, which updates the background model as new frames are processed and produces a foreground mask.

- Initialize the MOG2 background subtractor.
- Read selected frames from the traffic video.
- Apply the subtractor to each frame to obtain a foreground mask.
- Visualize the masks at different frame positions.

```
background_subtractor = cv2.createBackgroundSubtractorMOG2(
    history=500,
    varThreshold=25,
    detectShadows=True
)
foreground_mask = background_subtractor.apply(frame)
```

4.6 Motion Estimation Summary

Technique	Purpose	Observed Output
Frame Extraction	Read video frames for analysis	780 frames extracted
Frame Differencing	Highlight pixel changes between frames	Binary difference mask generated
Lucas-Kanade Optical Flow	Track feature-point movement	100 initial points; 100 successfully tracked
GMM / MOG2	Separate moving foreground from background	Foreground masks generated for sampled frames

5. Overall Experimental Results

The following table summarizes the numerical results that are directly visible in the provided Google Colab outputs.

Experiment	Observed Result
Traffic video inspection	25.0 FPS; 782 total frames; 1280 × 720 resolution; 31.28 seconds
Video frame extraction	780 frames extracted
Viola-Jones face detection	1 face detected
YOLOv8 object detection	6 car detections in the displayed traffic frame
Lucas-Kanade optical flow	100 initial feature points; 100 successfully tracked
Frame differencing	Foreground difference mask generated
GMM background subtraction	Foreground masks generated

6. Key Observations

- Viola-Jones detected the face in the uploaded portrait and displayed a bounding box around it.
- YOLOv8 identified multiple cars in the selected traffic frame and displayed class labels with confidence scores.
- The traffic video was successfully read, and 780 frames were extracted for further processing.
- Frame differencing highlighted changes between two consecutive grayscale frames.
- Lucas-Kanade optical flow successfully tracked all 100 feature points reported in the displayed output.
- GMM/MOG2 generated foreground masks at different sampled frames.

Note: Task 1 and Task 2 numerical metrics are intentionally left to be filled from the notebook's actual output. The document does not assume unverified PSNR, SSIM, HOG-dimension or SIFT match-count values.

7. Conclusion

This CCA 3 assignment demonstrated the implementation of several computer vision techniques using Python, OpenCV and Google Colab. The practical work covered image processing and segmentation, feature extraction and matching, object detection, and video motion estimation.

The object-detection experiments demonstrated the use of a Haar Cascade classifier for face detection and YOLOv8 for detecting vehicles in a traffic frame. The video-processing experiments demonstrated frame extraction, frame differencing, Lucas-Kanade optical flow and GMM/MOG2 background subtraction.

The outputs generated in Google Colab provide visual evidence of the implementation. The assignment helped develop practical understanding of how computer vision algorithms process images and video, identify objects, extract features and estimate motion.

Submission Details

Field	Details
Candidate Name	Sanket Salunkhe
PRN Number	1272250582
Course Module	Computer Vision – CCA 3
Primary Notebook	CCA_3_Computer_Vision_Assignment.ipynb
Execution Platform	Google Colab